Universidade Estadual de Santa Cruz (UESC)

Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação paralela em memória distribuída com MPI.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: André Nascimento Pardinho e Cauã Clemente Lima
Disciplina: DEC107 — Processamento Paralelo
Data: 30 de Outubro de 2026

1. Introdução

Essa etapa do projeto tem como objetivo analisar a aplicação do paralelismo em arquiteturas de memória distribuída, utilizando a interface MPI (Message Passing Interface), para otimizar o desempenho do cálculo do conjunto de Mandelbrot. Além da busca por aceleração, o estudo foca na avaliação de diferentes estratégias de decomposição do domínio da imagem e investiga o impacto dos custos de comunicação e coleta dos resultados no desempenho e na escalabilidade da aplicação

O conjunto de Mandelbrot é definido pelo conjunto de números complexos c para os quais a sequência iterativa $z_0 = 0$ e $z_{n+1} = z_n^2 + c$ permanece limitada. Computacionalmente, aplica-se o método de *escape-time*, no qual a recorrência é calculada para cada ponto do plano complexo até que o módulo de *z* ultrapasse o raio de escape (tipicamente 2) ou atinja um limite máximo de iterações. A quantidade de iterações realizadas determina o valor associado ao pixel e, consequentemente, sua representação na imagem do fractal.

A principal dificuldade computacionalmente do problema está na variação do custo de processamento entre os pixels. Pontos que escapam rapidamente exigem poucas interações, enquanto pontos que não ultrapassam o raio de escape dentro do limite podem exigir o número máximo de iterações. Essa variação também ocorre entre as linhas da imagem, produzindo diferentes cargas de trabalho para os processos responsáveis pelo cálculo.

Na Etapa 1 do projeto, o paralelismo foi implementado com OpenMP, utilizando threads que compartilham o mesmo espaço de memória e diferentes políticas de escalonamento para distribuir o trabalho. Nesta etapa, utiliza-se MPI, no qual cada processo possui seu próprio

espaço de memória e a troca de dados ocorre explicitamente por operações de comunicação. Essa característica exige que, além da distribuição dos cálculos, seja definida uma estratégia para reunir os resultados parciais e reconstruir a matriz da imagem.

Nesse sentido, foi adotado duas estratégias principais de distribuição: a decomposição estática em blocos contíguos de linhas e a distribuição cíclica de blocos de linhas. A comparação permite avaliar como a divisão das tarefas e a distribuição dos custos de processamento podem influenciar no balanceamento de carga e no tempo de execução. Também foi analisado o tempo de coleta de dados e reorganização dos resultados, visando compreender se esses fatores afetam o desempenho da implementação MPI e suas diferenças com a solução OpenMP da etapa anterior.

2. Metodologia

A implementação paralela em memória distribuída foi desenvolvida em linguagem C utilizando a biblioteca MPI. O código segue o modelo SPMD (Single Program, Multiple Data), no qual todos os processos executam o mesmo programa, mas trabalham sobre partes diferentes dos dados. A inicialização é realizada por MPI_Init, enquanto MPI_Comm_rank e MPI_Comm_size identificam, respectivamente, o processo que está sendo executado e a quantidade total de processo. Ao final da execução, MPI_Finalize encerra o ambiente MPI.

2.1 Estratégias de distribuição

A implementação mantém os parâmetros da versão sequencial na Etapa 1. O caso padrão considera a região completa do plano complexo com parte real Re ∈ [−2,0, 1,0] e parte imaginária Im ∈ [−1,5, 1,5], utilizando a resolução de 4096 x 4096 pixels e limite de 1000 interações. O caso de estresse, cavalos-marinhos, com zoom centrado em $(-\ 0,743643887;\ 0,131825904)$, com largura de $3,0 \times 10^{-3}$ no eixo real e limite de 5000 iterações. A resolução continua sendo configurável por argumento de execução.

Foram implementadas três opções de execução, que correspondem a duas estratégias principais de distribuição: A primeira estratégia foi a decomposição estática por blocos contíguos (bloco e blocov). A matriz da imagem é dividida horizontalmente em faixas de linhas atribuídas aos processos. Na variante ‘bloco’, todos os processos recebem a mesma

quantidade de linhas. Essa variante exige que a altura da imagem seja divisível pelo número de processos. Na variante ‘blocov’, a divisão considera o quociente e o resto da divisão entre a altura da imagem e a quantidade de processos. Quando a divisão não é exata, as linhas que sobram são distribuídas entre os primeiros processos, que recebem uma linha adicional.

A segunda estratégia foi a distribuição cíclica de blocos de linhas (intercaladov). A imagem é dividida em blocos contíguos de linhas, cujo tamanho é configurado pelo parâmetro ‘chunk’. Esses blocos são atribuídos estaticamente aos processos com base no resto da divisão de seu índice pelo número total de processos em execução. Assim, os processos recebem blocos alternados da matriz, em vez de sempre receberem uma única faixa contínua.

2.2 Coleta e reconstrução dos resultados

Cada processo calcula os pixels atribuídos à sua faixa ou aos seus blocos e armazena os resultados em um buffer local. Após o cálculo, os resultados são reunidos pelo processo raiz (rank 0). Na variante ‘bloco’, utiliza-se MPI_Gather, enquanto nas variantes ‘blocov’ e ‘intercaladov’ utiliza-se MPI_Gatherv. Nesta última, os dados recebidos são inicialmente armazenados em um buffer temporário, organizado de forma compacta por processo. Em seguida, o processo raiz reconstrói a ordem global das linhas por meio de cópias realizadas com memcpy, formando a matriz linear contínua utilizada na geração dos arquivos finais.

A escrita dos arquivos é realizada pelo processo raiz, que produz o arquivo binário .bin, contendo a contagem de iterações dos pixels, e uma imagem .ppm, utilizada para visualizar o fractal.

2.3 Ambiente de execução e automação dos testes

Os experimentos foram executados em um ambiente composto de um processador Ryzen 5 7535HS com 6 núcleos físicos e 12 processadores lógicos, 8GB de memória RAM e sistema operacional Zorin OS 18.1 (distribuição Linux baseada no Ubuntu 24.04 LTS).

As implementações foram compiladas com GCC 13.3.0, utilizando as opções de otimização -O3, -Wall, -Wextra e -std=c11. Para a versão OpenMP, foi adicionada a opção -fopenmp, enquanto a versão MPI foi compilada por meio do wrapper mpicc, utilizando Open MPI 4.1.6. A execução MPI foi realizada com mpirun -np N. Nos experimentos com oito

processos, foi utilizada a opção --use-hwthread-cpus, permitindo que o Open MPI considerasse os processadores lógicos disponíveis na alocação dos processos.

A automação dos experimentos, assim como da etapa 1, foi realizada por scripts em Python, responsáveis pela execução das configurações, repetição dos testes, coleta das métricas e armazenamento dos resultados em arquivos CSV separados por implementação e tipo de experimento. Para evitar a execução redundante da versão sequencial, foram geradas previamente referências seriais compartilhadas entre os benchmarks MPI e OpenMP. Cada combinação de caso e resolução foi executada cinco vezes, pegando a mediana dos tempos como referência para o cálculo do speedup. Os arquivos binários gerados também foram armazenados e reutilizados na verificação de corretude das implementações paralelas.

Os experimentos foram organizados em três grupos principais. Para a análise das estratégias MPI e do strong scaling, foi utilizado o caso padrão, com resolução de 4096 × 4096 pixels e limite de 1000 iterações. Foram avaliadas as estratégias bloco, blocov e intercaladov, com 1, 2, 4 e 8 processos MPI, realizando cinco repetições por configuração. Para comparação, a implementação OpenMP foi testada com as mesmas quantidades de unidades de processamento, utilizando as políticas static, dynamic e guided, todas com chunk de tamanho 1.

Para avaliar o balanceamento de carga na região de maior complexidade, foram realizados testes com o caso dos cavalos-marinhos, na resolução de 4096 × 4096 pixels e limite de 5000 iterações. Inicialmente, foram comparadas as três estratégias MPI com oito processos e chunk de tamanho 1. Em seguida, a estratégia intercaladov foi avaliada com chunks de 1, 16, 64 e 256 linhas. Experimentos equivalentes foram realizados com OpenMP, utilizando oito threads, as três políticas de escalonamento e os mesmos tamanhos de chunk.

Por fim, o weak scaling foi avaliado utilizando o caso padrão, com aumento proporcional da resolução e da quantidade de unidades de processamento. Na implementação MPI, foram utilizados os pares 4096 × 4096 com dois processos e 8192 × 8192 com oito processos. Na implementação OpenMP, foram utilizados os mesmos pares de resolução, com duas e oito threads, respectivamente. Em ambos os casos, foram mantidas as três estratégias MPI ou políticas OpenMP, com chunk de tamanho 1 e cinco repetições por configuração.

2.4 Procedimentos de medição de tempo

Para medir o desempenho, utilizou-se a função MPI_Wtime(). Antes do início da região cronometrada, os processos executam MPI_Barrier(MPI_COMM_WORLD), permitindo sincronizar o início da medição do cálculo. O tempo de cálculo é medido individualmente em cada processo, considerando o intervalo entre o início da computação local e seu término. Em seguida, operações MPI_Reduce, com os operadores MPI_MIN e MPI_MAX, identificam o menor e o maior tempo de cálculo dos processos.

O tempo da coleta é medido do processo raiz, desde antes da chamada do MPI_Gather ou MPI_Gatherv até o retorno da operação coletiva. Essa medida representa o tempo observado na fase de coleta, incluindo possíveis esperas por processos que ainda não chegaram à operação, não correspondendo exclusivamente ao tempo de transmissão dos dados. Na estratégia 'intercaladov', o tempo de reconstrução é medido separadamente.

O tempo paralelo total considera o intervalo observado pelo processo raiz desde antes da barreira inicial até a matriz global estar pronta na memória. Dessa forma, inclui a sincronização inicial, o cálculo, a coleta e a reorganização (quando aplicável), mas não considera a escrita dos arquivos. O tempo de I/O é registrado separadamente e corresponde à gravação dos arquivos binário e PPM.

2.5 Métricas de Desempenho

A avaliação considera tempo de execução, speedup, eficiência, fator de balanceamento de carga e escalabilidade forte e fraca. Para deixar mais claro os diferentes custos, são considerados tanto o speedup baseado no maior tempo local de cálculo quanto aquele baseado no tempo paralelo total. As duas medidas de speedup são mantidas separadas para distinguir a aceleração do cálculo matemático do desempenho da execução que também inclui coleta e reconstrução da matriz. O speedup de cálculo é definido por:

$$S_{calc} = \frac{T_{serial}}{T_{calc,max}}$$

Já o speedup da execução paralela completa, excluindo a escrita dos arquivos, é definido por:

$$S_{total} = \frac{T_{serial}}{T_{paralelo,total}}$$

A eficiência correspondente é calculada dividindo-se o speedup pelo número de processos MPI:

$$E_{total} = \frac{S_{total}}{P}$$

em que P representa a quantidade de processos utilizados. O desbalanceamento de carga é avaliado pelo fator F_{LB}, calculado a partir dos tempos locais de cálculo:

$$F_{LB} = \frac{T_{\max} - T_{\min}}{T_{\max}}$$

em que T_{max} e T_{min} correspondem ao maior e ao menor tempo de cálculo entre os processos MPI. A métrica considera somente o tempo de computação local, sem incluir a coleta ou a reorganização dos dados. Um valor igual a zero indica tempos de cálculo iguais entre os processos, enquanto valores próximos de 1 indicam maior diferença entre os tempos locais.

3. Resultados e Discussões

Os arquivos .csv's gerados nos casos de teste descritos anteriormente mostram que a corretude em relação ao código serial está adequada, tendo diferença de 0 pixels em todos os 3 tipos de testes principais:

Figura 1 - Corretude do caso de teste Strong Scaling

caso	resolucao	max_iter	estrategia	repeticao	processos	correto_exato
padrao	4096	1000	bloco	1	1	SIM
padrao	4096	1000	bloco	2	1	SIM
padrao	4096	1000	bloco	3	1	SIM
padrao	4096	1000	bloco	4	1	SIM
padrao	4096	1000	bloco	5	1	SIM
padrao	4096	1000	blocov	1	1	SIM
padrao	4096	1000	blocov	2	1	SIM
padrao	4096	1000	blocov	3	1	SIM
padrao	4096	1000	blocov	4	1	SIM
padrao	4096	1000	blocov	5	1	SIM
padrao	4096	1000	intercaladov	1	1	SIM
padrao	4096	1000	intercaladov	2	1	SIM
padrao	4096	1000	intercaladov	3	1	SIM
padrao	4096	1000	intercaladov	4	1	SIM
padrao	4096	1000	intercaladov	5	1	SIM
padrao	4096	1000	bloco	1	2	SIM
padrao	4096	1000	bloco	2	2	SIM
padrao	4096	1000	bloco	3	2	SIM
padrao	4096	1000	bloco	4	2	SIM
padrao	4096	1000	bloco	5	2	SIM
padrao	4096	1000	blocov	1	2	SIM
padrao	4096	1000	blocov	2	2	SIM
padrao	4096	1000	blocov	3	2	SIM

Figura 2 - Corretude do caso de teste strategies_cavalos_chunks

caso	resolucao	max_iter	estrategia	repeticao	processos	correto_exato
cavalos	4096	5000	bloco	1	8	SIM
cavalos	4096	5000	bloco	2	8	SIM
cavalos	4096	5000	bloco	3	8	SIM
cavalos	4096	5000	bloco	4	8	SIM
cavalos	4096	5000	bloco	5	8	SIM
cavalos	4096	5000	blocov	1	8	SIM
cavalos	4096	5000	blocov	2	8	SIM
cavalos	4096	5000	blocov	3	8	SIM
cavalos	4096	5000	blocov	4	8	SIM
cavalos	4096	5000	blocov	5	8	SIM
cavalos	4096	5000	intercaladov	1	8	SIM
cavalos	4096	5000	intercaladov	2	8	SIM
cavalos	4096	5000	intercaladov	3	8	SIM
cavalos	4096	5000	intercaladov	4	8	SIM
cavalos	4096	5000	intercaladov	5	8	SIM

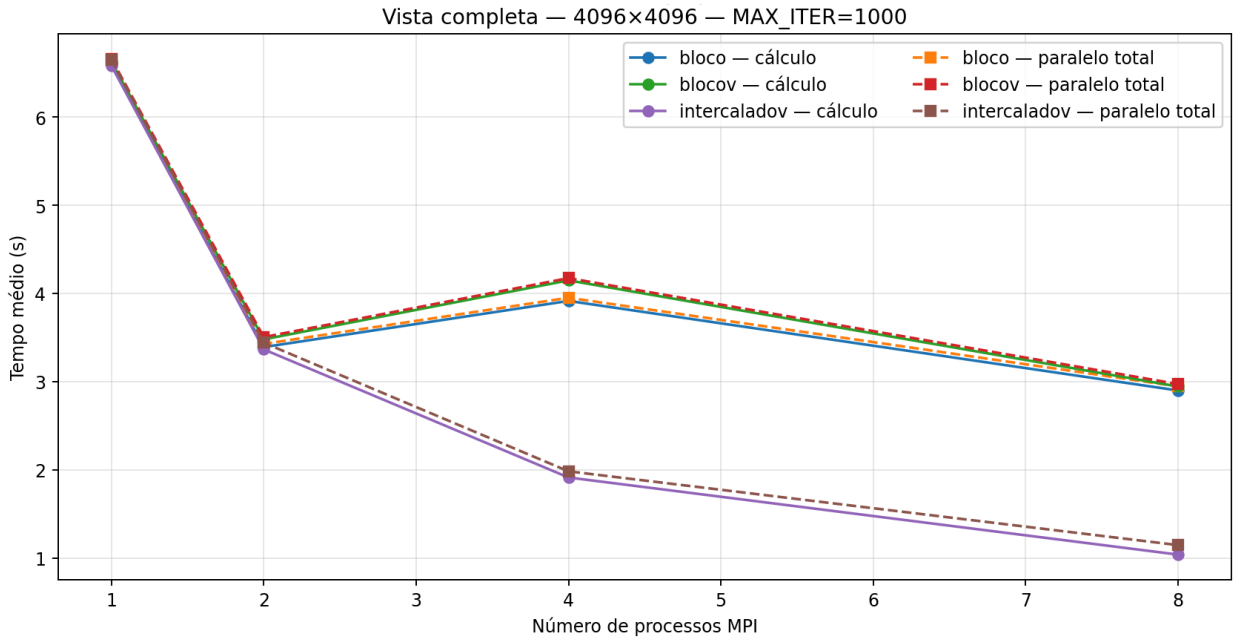
Figura 3 - Corretude do caso de teste weak_scaling

caso	resolucao	max_iter	estrategia	repeticao	processos	correto_exato
padrao	4096	1000	bloco	1	2	SIM
padrao	4096	1000	bloco	2	2	SIM
padrao	4096	1000	bloco	3	2	SIM
padrao	4096	1000	bloco	4	2	SIM
padrao	4096	1000	bloco	5	2	SIM
padrao	4096	1000	blocov	1	2	SIM
padrao	4096	1000	blocov	2	2	SIM
padrao	4096	1000	blocov	3	2	SIM
padrao	4096	1000	blocov	4	2	SIM
padrao	4096	1000	blocov	5	2	SIM
padrao	4096	1000	intercaladov	1	2	SIM
padrao	4096	1000	intercaladov	2	2	SIM
padrao	4096	1000	intercaladov	3	2	SIM
padrao	4096	1000	intercaladov	4	2	SIM
padrao	4096	1000	intercaladov	5	2	SIM
padrao	8192	1000	bloco	1	8	SIM
padrao	8192	1000	bloco	2	8	SIM
padrao	8192	1000	bloco	3	8	SIM
padrao	8192	1000	bloco	4	8	SIM
padrao	8192	1000	bloco	5	8	SIM
padrao	8192	1000	blocov	1	8	SIM
padrao	8192	1000	blocov	2	8	SIM
padrao	8192	1000	blocov	3	8	SIM

Comprovado a corretude, passamos para a análise de desempenho nos casos de teste.

AVALIAÇÃO DE TEMPO NO MPI:

Figura 4 - Tempo de cálculo e tempo paralelo total (MPI)



A partir dos resultados obtidos, pode-se notar que as estratégias de bloco e blocov possuem o tempo de cálculo extremamente próximos com o tempo paralelo total, enquanto que a estratégia de intercaladov possui uma pequena mas visível diferença decorrente do tempo de reorganização da matriz no processo de rank 0 e da coleta dos resultados . Devido à diferença que foi considerada mínima, o fator de balanceamento calculado nos testes seguintes foram realizados levando em conta apenas o tempo de cálculo, sem considerar o tempo de reorganização que só se aplica no intercaladov.

AVALIAÇÃO DO TESTE STRONG SCALING:

Figura 5 - Tempo médio de execução paralela (MPI x OpenMP)

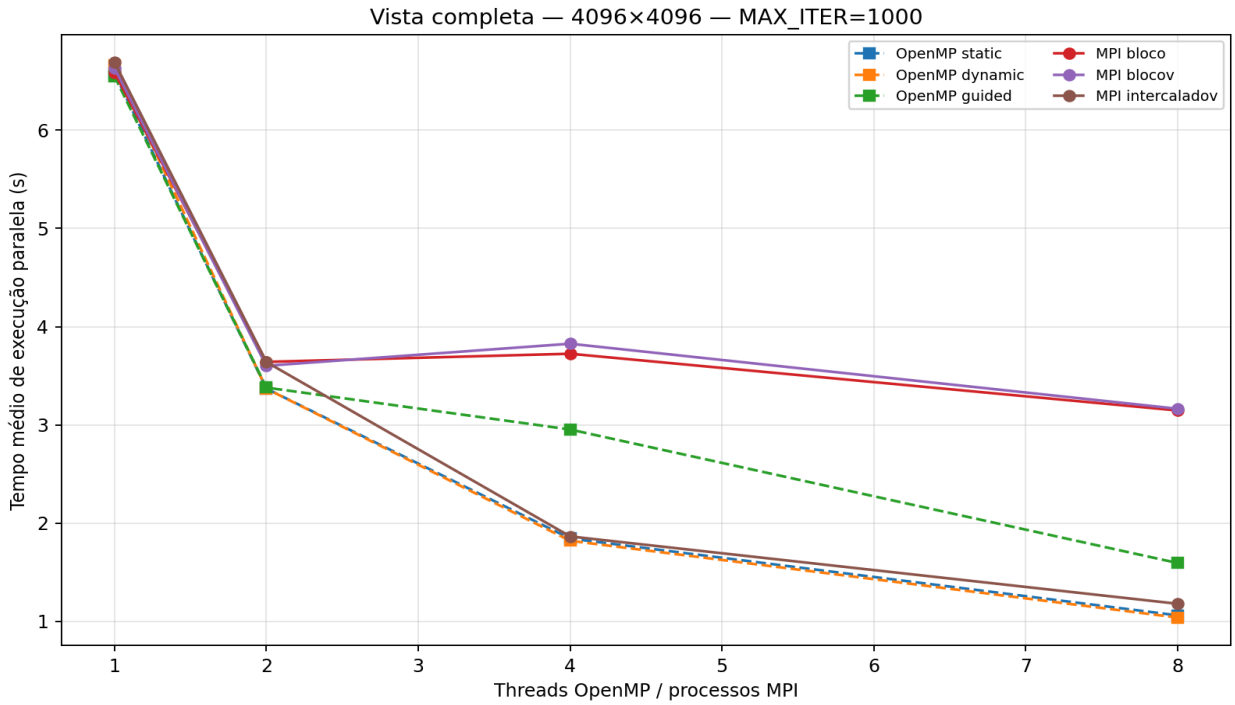


Figura 6 - Speedup de cálculo (MPI x OpenMP)

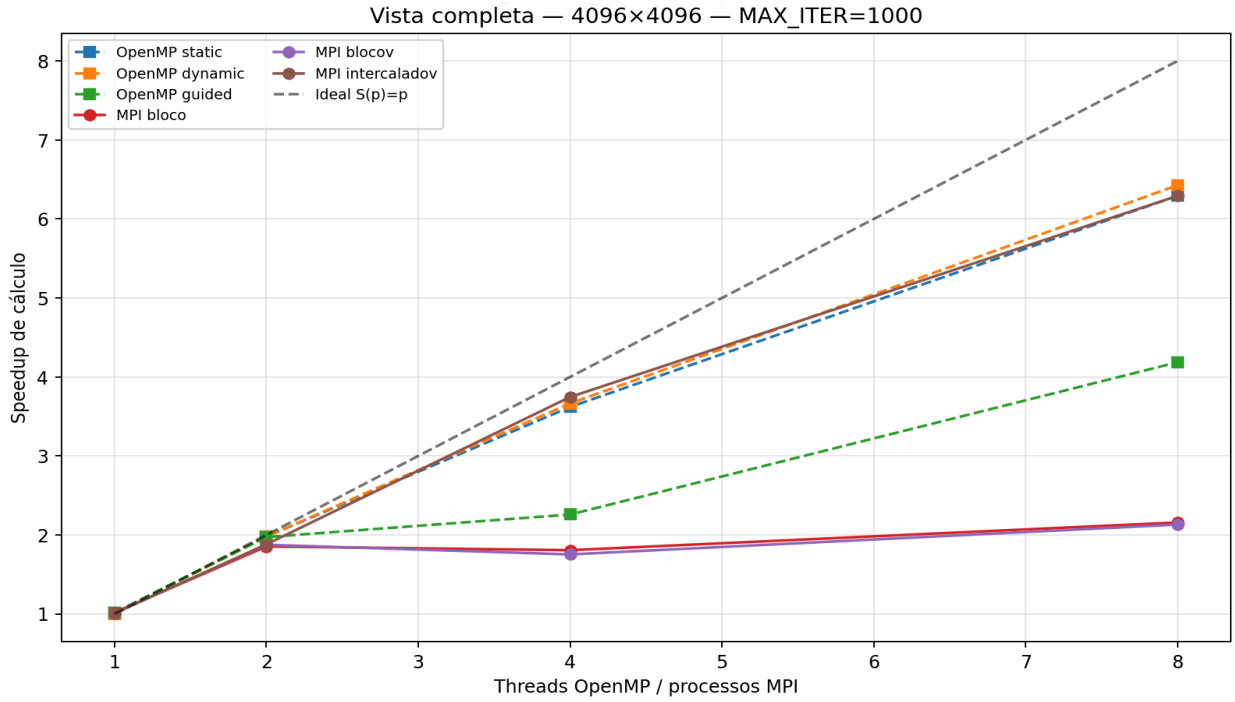
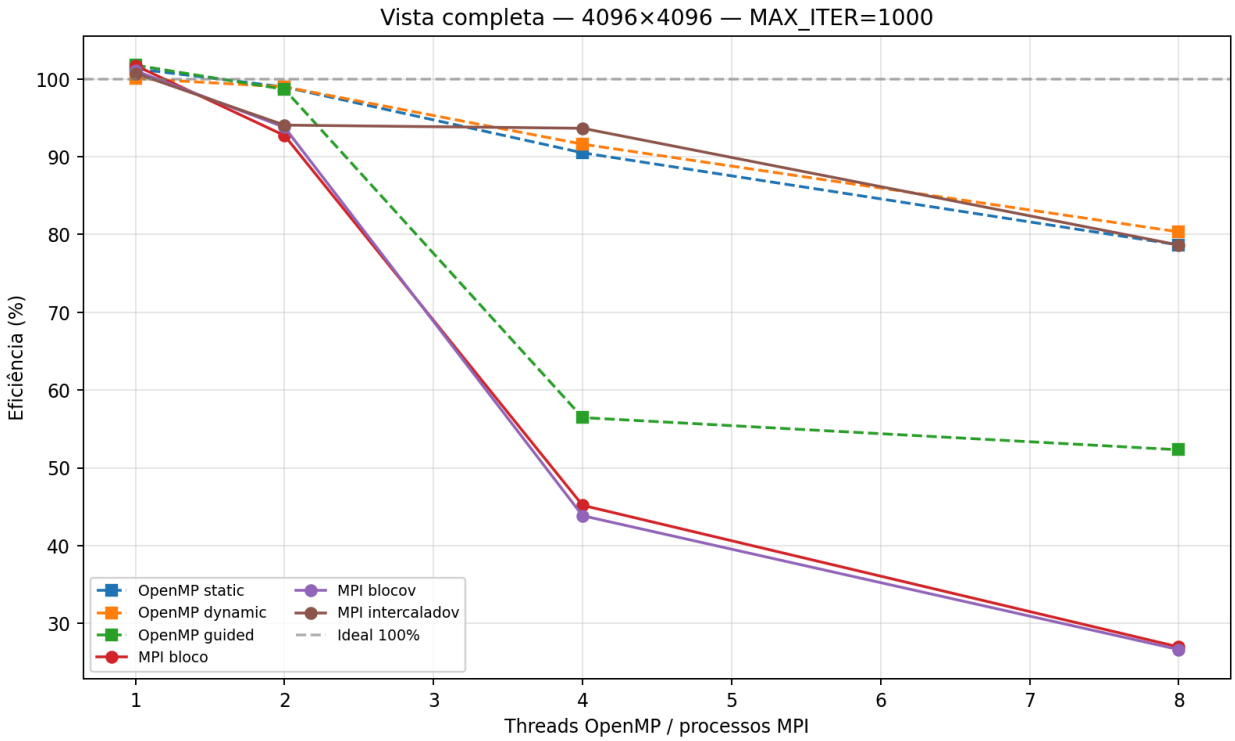


Figura 7 - Eficiência de cálculo (MPI x OpenMP)



Podemos perceber a partir dos gráficos gerados a partir dos *.csv's* resultantes que, conforme a adição de processos para divisão do trabalho, o tempo de cálculo diminui e o speedup em relação ao serial aumenta, como esperado, com a única exceção sendo as estratégias de bloco e blocov com 4 processos, sendo decorrente do desbalanceamento. Ainda sobre essas duas estratégias, percebe-se que, ainda que sejam melhores que as versões seriais, são muito inferiores às outras estratégias e schedules, o que já era esperado. A estratégia de intercaladov, no entanto, se mostrou bastante eficaz, comparando-se às schedules de static e dynamic do OpenMP.

Assim como no projeto anterior feito com OpenMP, vale notar que, apesar da melhora, cada adição consecutiva de threads diminui a melhora em relação à adição anterior, relacionado à lógica da Lei de Amdahl.

COMPARAÇÃO DE ESTRATÉGIAS NO VALE DOS CAVALOS COM CHUNK = 1:

Figura 8 - Cálculo e tempo paralelo total (MPI)

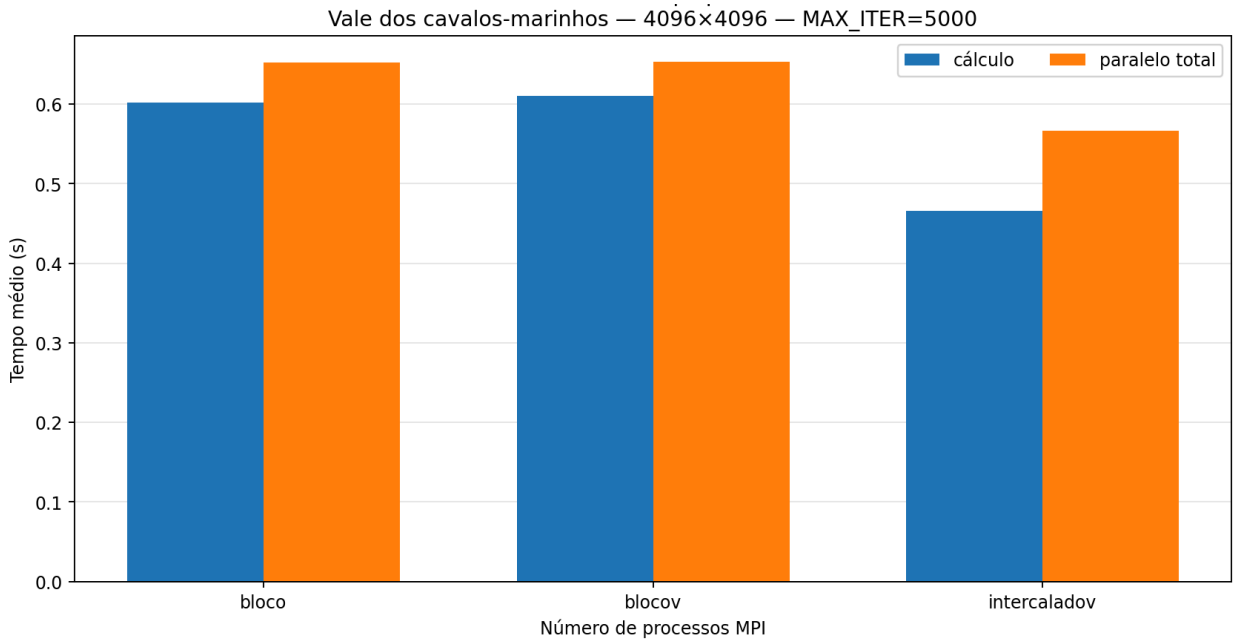
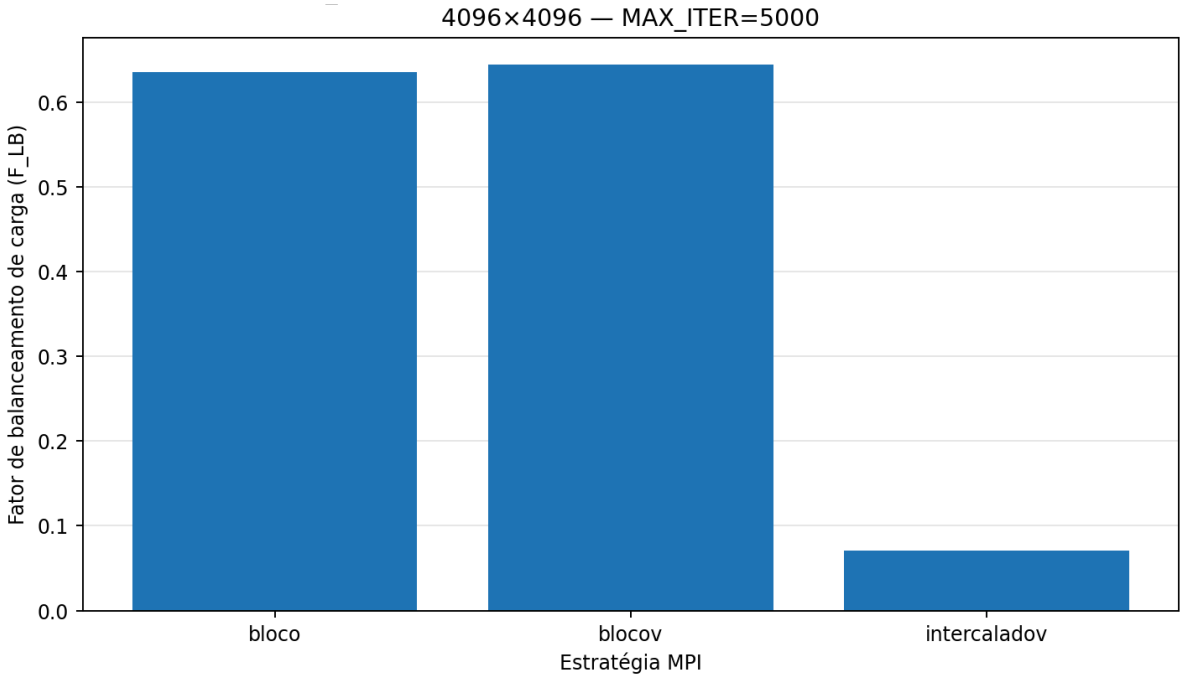


Figura 9 - F_LB por estratégia no Vale dos Cavalos-Marinhos (MPI)



Pode-se notar que a estratégia intercaladov apresentou o menor tempo de cálculo em relação a bloco e blocov. Mesmo considerando o custo adicional de coleta e reorganização da matriz, a distribuição cíclica também apresentou o menor tempo paralelo total.

Na figura 9 é possível notar que enquanto as estratégias contíguas apresentaram fatores de balanceamento próximos de 0,64, a estratégia intercaladov obteve aproximadamente 0,07, indicando uma distribuição muito mais uniforme do trabalho entre os processos. Assim, a redução do tempo de cálculo compensou o custo adicional de reorganização dos resultados.

Também é importante observar que, embora o caso dos cavalos-marinhos utilize um limite de 5000 iterações, superior ao limite de 1000 da vista completa, seus tempos de execução foram menores. Uma possível explicação para isso seria que o limite máximo não representa necessariamente a quantidade de iterações executadas por cada pixel. O custo efetivo depende da quantidade de iterações realizadas até a condição de escape, que varia conforme a região do plano complexo.

COMPARAÇÃO DE ESTRATÉGIAS/SCHEDULES E CHUNKS:

Figura 10 - Tempo médio de execução paralela por chunk (MPI x OpenMP)

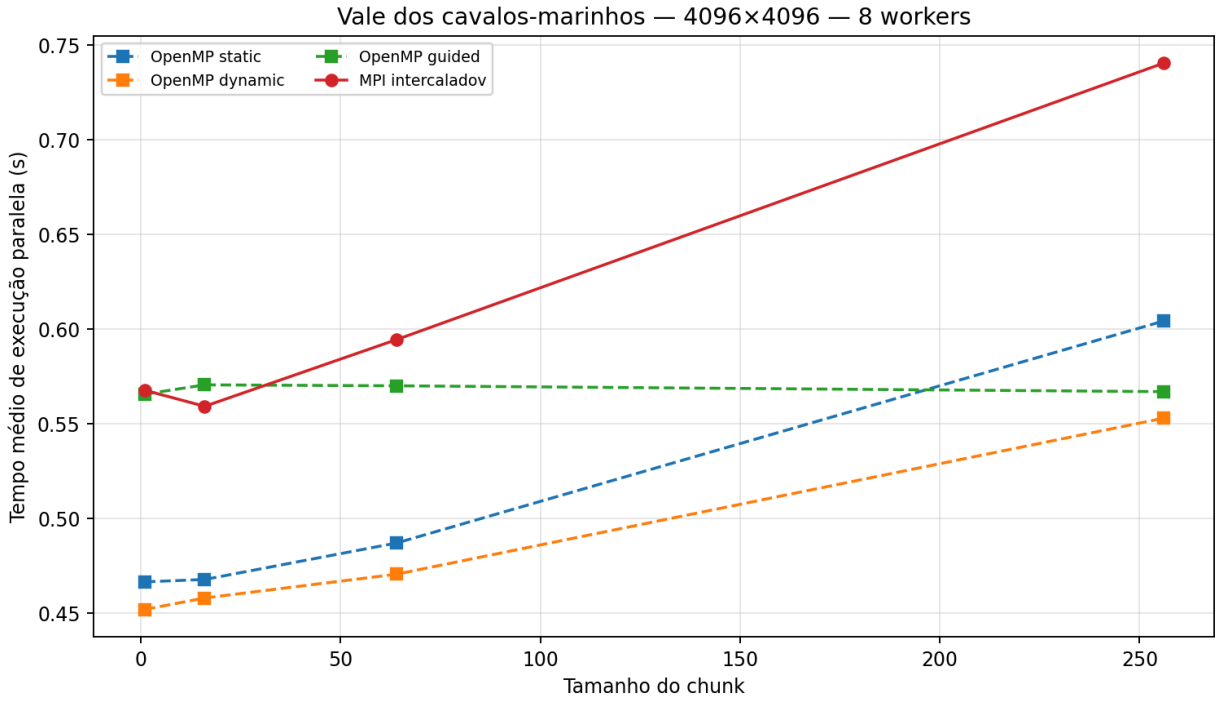


Figura 11 - Fator de balanceamento de carga por chunk (MPI x OpenMP)

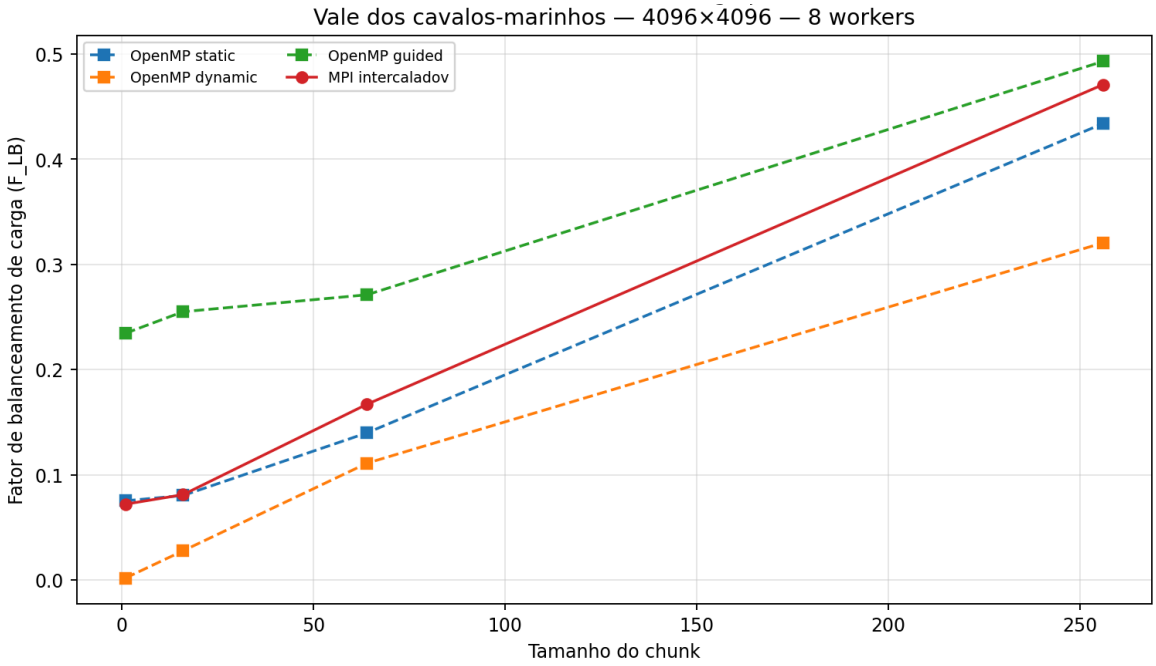
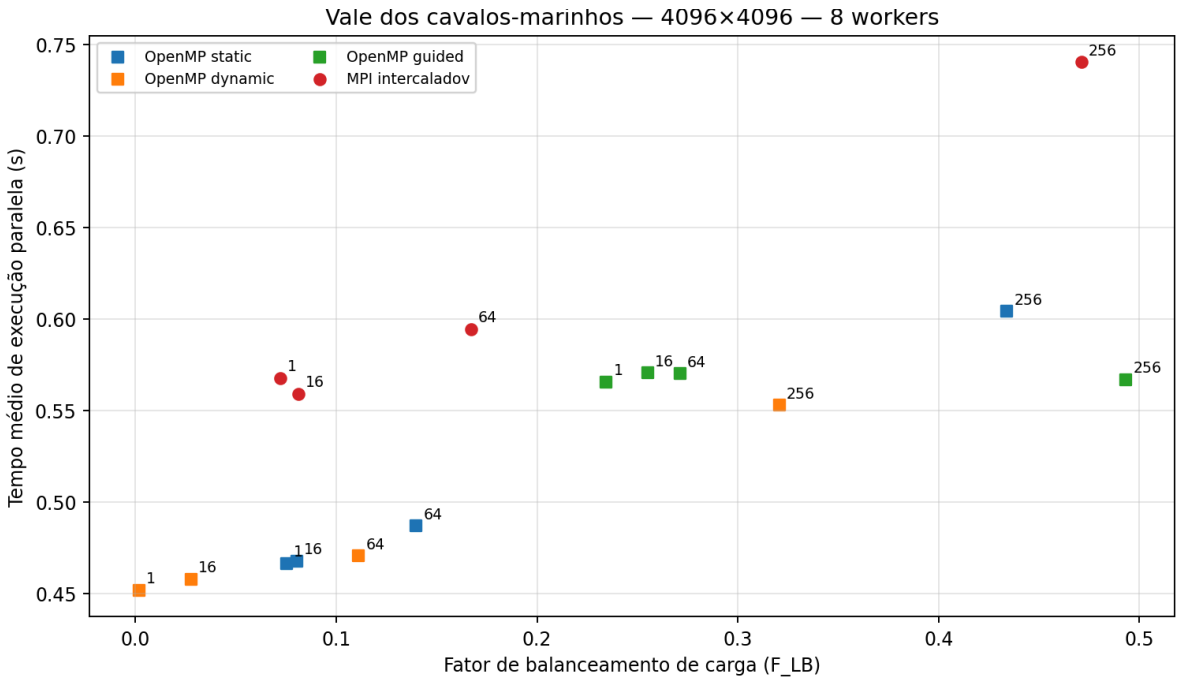


Figura 12 - F_LB e Tempo de execução paralela (MPI x OpenMP)

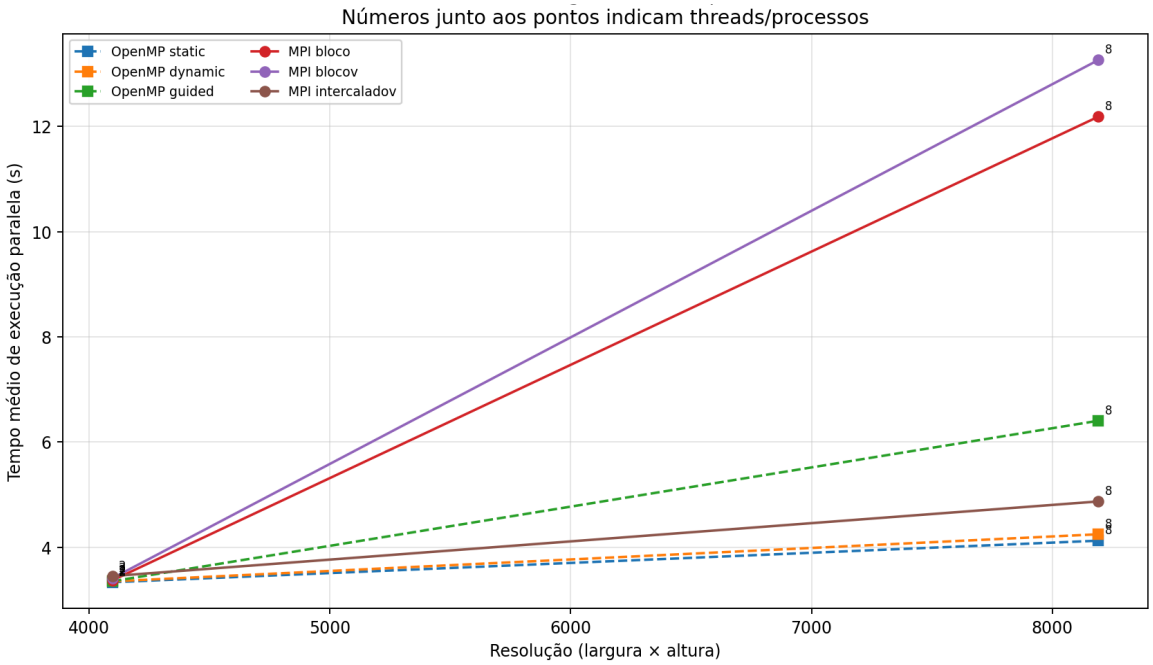


A partir dos testes realizados com intercaladov, nota-se que ele tem um desempenho semelhante ao schedule guided no vale dos cavalos marinhos, mas perde muito a eficiência conforme o aumento do tamanho dos chunks, além de piorar o fator de balanceamento.

Como o intercaladov funciona semelhante ao static, esperava-se que obtivesse um desempenho de tempo semelhante. Apesar disso, é importante notar que, mesmo que o desempenho em tempo tenha diferido, o fator de balanceamento de carga dos dois seguiu um crescimento semelhante.

AVALIAÇÃO DO TESTE WEAK SCALING:

Figura 13 - Weak Scaling na Vista Completa (MPI x OpenMP)



Nesse teste, foi possível notar que o aumento da resolução de 4096 para 8192, juntamente com o aumento de dois para oito processos, não manteve o tempo de execução constante nas implementações. Nas estratégias bloco e blovov do MPI, houve um aumento no tempo paralelo total que indica que a divisão em faixas contíguas pode apresentar dificuldades de balanceamento à medida que aumenta a quantidade de processos, pois a distribuição custo não é uniforme. Por outro lado, a estratégia intercaladov apresentou um aumento mais moderado. Esse resultado demonstra a vantagem da distribuição cíclica de linhas, que espalha regiões de diferentes custos computacionais entre os processos, reduzindo o impacto do desbalanceamento.

Nas implementações OpenMP, as políticas static e dynamic também apresentaram resultados relativamente próximos, enquanto guided alcançou um tempo um pouco maior.

Dessa forma, static e dynamic apresentaram os comportamentos mais próximos da escalabilidade ideal nesse experimento.

De maneira geral, a estratégia intercaladov apresentou uma escalabilidade mais favorável do que as distribuições MPI contíguas, aproximando-se dos resultados obtidos pelas políticas static e dynamic do OpenMP.

4. Conclusão

A segunda etapa do projeto permitiu aprofundar o estudo do paralelismo em arquiteturas de memória distribuída por meio da implementação de MPI para o cálculo do conjunto de Mandelbrot. Foi possível comparar diferentes estratégias de distribuição do trabalho e analisar o comportamento da aplicação em cenários de escalabilidade forte e fraca.

Entre as estratégias MPI avaliadas, a distribuição cíclica de blocos de linhas (intercaladov) apresentou os resultados mais favoráveis nos cenários de maior desbalanceamento. No caso dos cavalos-marinhos, essa estratégia obteve um fator de balanceamento consideravelmente menor que as distribuições contíguas, além de reduzir o tempo de execução mesmo considerando o custo adicional de coleta e reorganização da matriz. Os testes de chunks também demonstraram que blocos menores favoreceram o balanceamento, enquanto o aumento da granularidade prejudicou o desempenho da distribuição cíclica.

Na comparação com OpenMP, as políticas static e dynamic apresentaram resultados melhores no strong scaling e no weak scaling, enquanto dynamic obteve os melhores resultados no teste de variação de chunks para o caso dos cavalos-marinhos. A estratégia intercaladov aproximou-se do desempenho dessas políticas em alguns cenários, mostrando que uma distribuição cíclica pode ser eficaz mesmo em um modelo de memória distribuída. Por outro lado, as estratégias MPI contíguas foram mais sensiveis ao desbalanceamento, especialmente no weak scaling.

Por fim, outro ponto observado foi que o aumento da quantidade de processos ou threads não garante ganhos proporcionais de desempenho. Além dos custos de comunicação e coleta no MPI, a distribuição irregular do trabalho e as limitações do hardware influenciam diretamente na eficiência obtida.

5. Referências

- CET107 - Processamento Paralelo Aula 10 - MPI
- CET107 - Processamento Paralelo Aula 11 - MPI
- https://pt.wikipedia.org/wiki/Lei_de_Amdahl
- https://www.open-mpi.org/doc/

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, André Nascimento Pardinho, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “parallel-processing-mandelbrot", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.
(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: ChatGPT 6, Gemini e GitHub Copilot. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Auxílio na depuração (debugging) de blocos específicos do código da implementação paralela
- Geração de scripts base para automação de testes de desempenho como o benchmark_mpi.py e o gerar_referencias.py;

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 30/10/2026
André Nascimento Pardinho
Matrícula: 202410507

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Cauã Clemente Lima, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “parallel-processing-mandelbrot”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.
(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: ChatGPT 6. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Geração de scripts base para utilização da função GatherV nas estratégias de blocov e intercaladov;
- Geração de scripts para criação de gráficos referentes aos resultados obtidos nos experimentos;
- Auxílio na depuração (debugging) de blocos específicos do código da implementação da estratégia de intercaladov e correção do código de gráfico

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 30/10/2026
Cauã Clemente Lima
Matrícula: 202410120